Over-The-Air Update Service

User Guide

SUMMARY: This document provides the functional specification and user guide for the Atmosic Wireless SoC Series Over-The-Air (OTA) Bluetooth Low Energy server profile. The OTA profile can be integrated into an application to provide reliable application software updates. The updates can be delivered via a Bluetooth LE client operating on a remote central device.

CONFIDENTIAL



Atmosic™

Table of Contents

Acronyms and Abbreviations **2**
1. Overview **3**
2. Features **4**
2.1 New to Version 1.4 6
3. Design **6**
3.1 Architecture 6
3.1.1 Client Profile 6
3.1.2 Server Profile 7
3.1.3 Upgrade Processor 9
3.1.4 Storage Subsystem 9
3.1.5 RRAM / External Flash 9
3.1.6 MCUBoot 11
3.1.7 Upgrade State 11
3.2 Upgrade Process Flow 12
3.2.1 Use of a CRC/Hash 12
3.2.2 No Test Boot Option 12
3.2.3 Upgrade Flow Steps 14
3.2.4 First Boot Watchdog 15
3.3 Messages 16
3.3.1 Types 16
Command 16
Status 16
Completion 16
Write Data 16
3.3.2 Ports 17
Mailbox 17
Bulk 17
3.3.3 Definitions 17
Message Format 17
Command Status 18
Command Completion 19
Cancel 19
Bluetooth LE Considerations 19
Message size limits 19
Pending Message Indication 20
Locking and Unlocking Sessions 20
Lock Session 22
Unlock Session 22
Query Information 22
Query Version Information 24
Write Data Start 24
Write Data Continuation 26
Last Write Status 27

CONFIDENTIAL

Read Data 27
Get Programmed State 28
Erase Sectors 29
Set Upgrade Status 29
Reboot 30
3.4 Message Flows 31
3.4.1 Control 31
3.4.2 Write Data 31
3.4.3 Write Data Stream Mode 31
4. Developers Guide 35
4.1 Installing the profile 35
4.1.1 Board Identifier 35
4.1.2 Write Buffer Size 35
4.1.3 First boot timeout 35
4.1.4 Version String 36
4.2 Changing the Service UUID 37
4.3 Configuring the Slot Sizes 37
4.4 Cleaning Upgrades 37
4.5 Testing with a Non-Upgradeable Partition Scheme 38
Reference Documents 38
Revision History 39

CONFIDENTIAL

List of Figures

Figure 1 - OTA System Block Diagram
Figure 2 - RRAM / Frash Subsystem Block Diagram
Figure 3 - Image Upgrade Flow Diagram
Figure 4 - Command/Status/Completion Flow Diagram
Figure 5 - Write Data Flow for Mailbox Port
Figure 6 - Write Buffer Fill with a Dropped Message
Figure 7 - Write Data Flow for Bulk Port

List of Tables

Table 1 - Attribute Definition
Table 2 - Message Format
Table 3 - Status Codes

Acronyms and Abbreviations

Acronym	Definition
ATT	Attribute Protocol
BULK	DTS bulk data port
CCC	Client Characteristic Configuration
CRC	Cyclic Redundancy Check
DIS	Device Information Service
DTS	Data Transfer Service
GATT	Generic Attribute Profile
MBOX	DTS mailbox data port
MCU	Microcontroller Unit
MTU	Maximum Transmission Unit
NVDS	Non-Volatile Data Store
OTA	Over-the-Air
PDU	Protocol Data Unit
RFU	Reserved for Future Use
ROM	Read-Only Memory
RRAM	Resistive Random Access Memory
UUID	Universally Unique Identifier

CONFIDENTIAL

1. Overview

OTA updates allow end users to update their systems in the field without the need for wired connections. A mobile application can deliver firmware updates wirelessly as part of the device onboarding process or a regularly scheduled update process. This document describes the Atmosic proprietary OTA update Bluetooth LE profile (server/peripheral role) and messaging interface to allow users to manage upgrades through mobile or desktop client applications. This document pertains to the unique use requirements when using internal RRAM and external flash memories available in ATM33/e designs. Previous generations such as the ATM2/ATM3 series can continue to use the previous OTA protocols. See Reference Documents section for related documents.

2. Features

- OTA server profile with customizable 128-bit service UUID.
- Server profile can coexist with other profiles.
- Profile can be installed dynamically or statically at boot time.
- Low memory footprint with application configurable write buffer size.
- Supports Atmosic MCUBoot loader and upgradeable image partitioning scheme.
- Hybrid storage for MCUBoot systems where external SPI flash provides data storage for upgrades instead of RRAM.
- Support for optional first boot watchdog mechanism
- Flexible OTA command set to develop both simple and complex (background update) client applications.
- Allows resumed updates from connection or power cycling interruptions.
- Standard Bluetooth LE link security and authentication
- Optional performance mode for write data transfers using the DTS bulk port data flow.

2.1 New to Version 1.4

The upgrade protocol version 1.4 is backwards compatible with the previous version of the protocol (v1.0-1.3) supporting both legacy ATM2 flash based upgrades and ATM33 RRAM/MCUboot based upgrades. The following section lists additions and new restrictions :

- Extension to Query Information

CONFIDENTIAL

    - A new capability flag to indicate no-test boot support (MCUBoot overwrite strategy)
    - A new capability flag to indicate a HYBRID storage arrangement where the inactive slot is managed in external flash.
    - A new 16-bit extended capabilities field has been added for future expansion.

- Boot Flow Changes
    - Hybrid configuration using external flash requires erase operations.
    - **The active partition can no longer be accessed in an MCUBoot configuration (only in a legacy ATM2 configuration). This will disallow the use of READ, GET_PROGRAM_STATE and COPY commands targeting the active partition.**
    - **For systems that utilize external flash for OTA storage, the lock session command must be used to ensure that the flash storage device is powered. Operations can fail if the underlying flash storage device is powered down.**
- Features officially deprecated for ATM33 (MCUBoot based)
    - Copy command
    - NVDS reading/copying
    - READ and GET_PROGRAM_STATE commands targeted at the active partition.

CONFIDENTIAL

3. Design

3.1 Architecture

The OTA system is depicted in Figure 1:

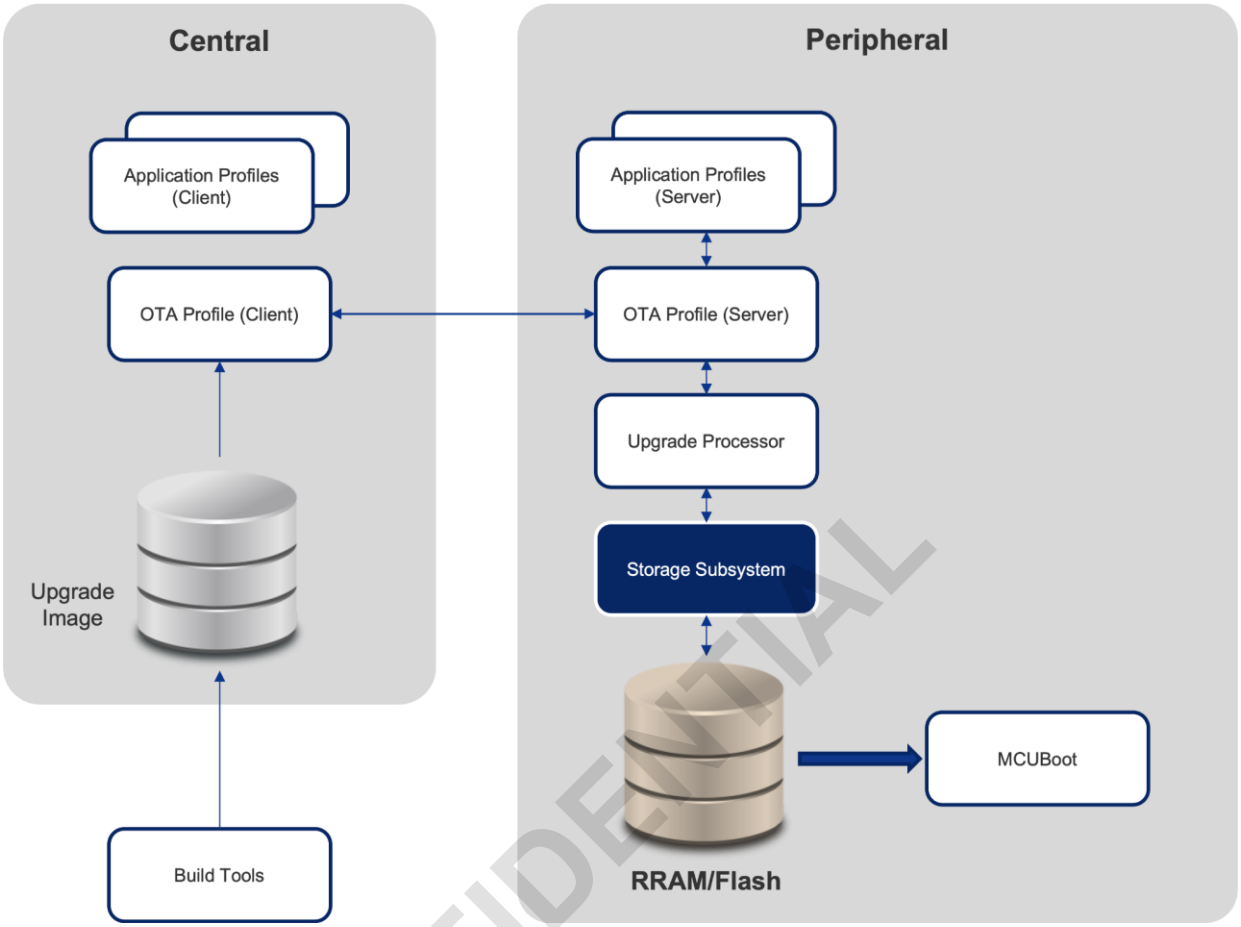


Figure 1 - OTA System Block Diagram

CONFIDENTIAL

The central is the mobile or desktop host running the end-user application. Atmosic provides client libraries for common operating systems to rapidly integrate OTA capabilities into your application. The following section provides details on each of the major components.

3.1.1 Client Profile

The client profile is implemented in the central's Bluetooth LE stack (mobile or desktop stack). The profile is GATT-based and will use GATT write and read procedures to communicate with server-side characteristics. The client interface to the OTA system consists of DTS ports that implement communications ports tailored for command/response and data.

3.1.2 Server Profile

The server profile implements the OTA upgrade message command set and data transfer protocols. The server leverages the data transfer service (DTS) ports and data flow behavior and requires the following port types:

- 1 bulk port (optional fast write data transfers) - 512 byte wide
- 1 mailbox port (control/status/data) - 512 bytes wide

The default service UUID is: a0e78d39-75b5-4182-8fdc-c4b7365c9062
Customers can reconfigure the profile to install their own UUID.

Attribute	Attribute Type	Permission	Values	Notes
Service	Primary Service Declaration	RO	Service UUID defined by the user	No include declaration needed. UUID: default ATM OTA
Bulk Port Char	Characteristic Declaration	RO	Properties, Value Handle, UUID	UUID: 040C6507-F08A-4FC2-ABEA-6382EE250230
Bulk Port Char Value	Characteristic Value Declaration	Write No-RSP / Notify	512 -byte value size	Bulk Data Flow (Stream). Max transfer ATT_MTU - 3
Bulk Port CCC	Client Characteristic Configuration Descriptor	R/W	CCC bit	Set value to 0x0001 to enable ATT notifications
MBOX Port Char	Characteristic Declaration	RO	Properties, Value Handle, UUID	UUID: CB91B0D6-3080-4DFB-876B-407DB045E52B
MBOX Port Char Value	Characteristic Value Declaration	Write / Indicate	512 -byte value size	Mailbox Data Flow. Max read/write of 512 bytes. Max indicate is ATT_MTU - 3
MBOX Port CCC	Client Characteristic Configuration Descriptor	R/W	CCC bit	Set value to 0x0002 to enable ATT indications

Table 1 - Attribute Definition

CONFIDENTIAL

3.1.3 Upgrade Processor

The upgrade processor is a reusable library that implements the image upgrade command set. There is only one instance of the upgrade processor in the system and it will be initialized and managed by the server profile. The Bluetooth LE server profile provides an over-the-air messaging channel that bridges the client to the upgrade

processor. For most commands the Bluetooth LE client is interacting mainly with the upgrade processor. The upgrade engine interfaces with the storage subsystem which provides access to either RRAM and/or FLASH storage media.

3.1.4 Storage Subsystem

This consists of the storage subsystem (drivers) that manage the underlying storage medium. The upgrade processor uses an upgrade storage interface to support internal RRAM and optionally an interface to support external SPI flash devices when configured. Please refer to the user's guide for details on partitioning schemes when using external flash to provide the storage for upgrades.

3.1.5 RRAM / External Flash

The memory storage subsystem is divided into areas containing code and data as shown in Figure 2:

CONFIDENTIAL

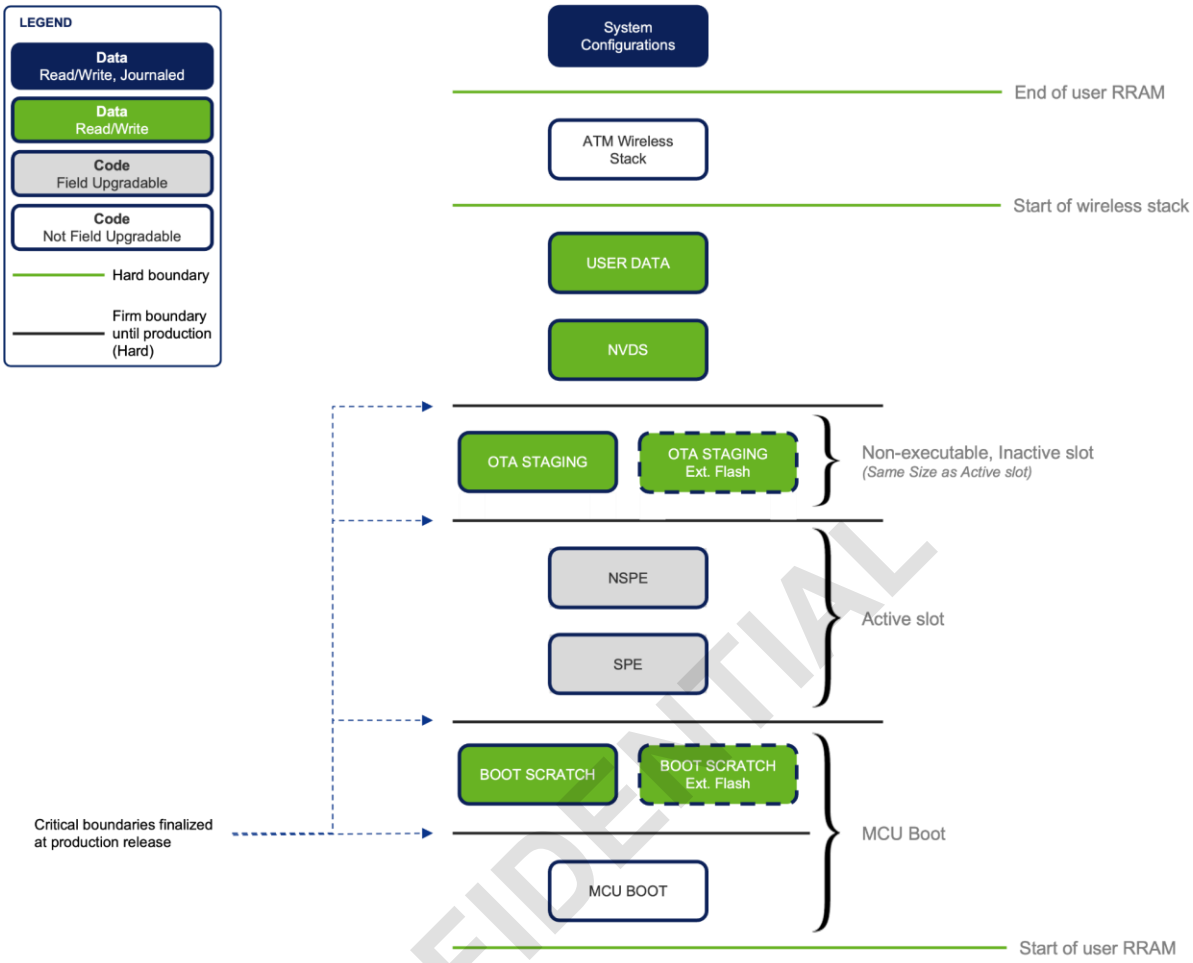


Figure 2 - RRAM / Flash Subsystem Block Diagram

CONFIDENTIAL

The OTA server allows access to the OTA staging area (read/write) and in non-MCUBoot configurations, access to NVDS (read only) and the active image area (read only). **In an MCUBoot configuration access to the active image area is no longer allowed**. In systems with RRAM only, the OTA staging and boot scratch data areas also reside in RRAM. In a hybrid configuration where external flash is used to augment storage, these areas are located in the external flash device. This provides considerably more RRAM for application use. RRAM is always used as the active slot that runs the application code. External flash, if used, is only used in data storage mode for the staging area and boot scratch and represents the inactive slot/partition. Details of each area is described in the **ATM33/e Wireless SoC Series SDK User Reference Manual**. In previous external flash-based products, the partitions were binary (half the storage device size) with sub-partitions laid out for NVDS and user data. For ATM33, the partitions are segmented to support reliable image upgrades with provisions for user data and NVDS. In ATM33 the upgrade partition term is synonymous with MCUBoot's slot nomenclature. In previous generations the OTA server protocol referred to these as active/inactive

upgrade partitions. For further discussions, an active slot is equivalent to an active partition and the inactive slot is equivalent to the inactive partition.

In a hybrid configuration the information returned in the query information command refers to the external flash device managing the inactive slot/partition. The active partition is inaccessible and remains hidden. All operations target the inactive partition.

3.1.6 MCUBoot

The system boot chain begins with the ROM and transfers control to a primary bootloader stage in RRAM. The current primary bootloader is based on the open source MCUboot project. This bootloader is non-upgradeable and is only used to validate the currently active image for integrity and optionally authenticate the image (secure boot). The primary bootloader is also tasked to detect an image update request and swap images between the active and inactive slots. A recovery mechanism is employed that detects a failed boot or an unacknowledged boot (first boot test that fails to be acknowledged) and will automatically swap the original image back.

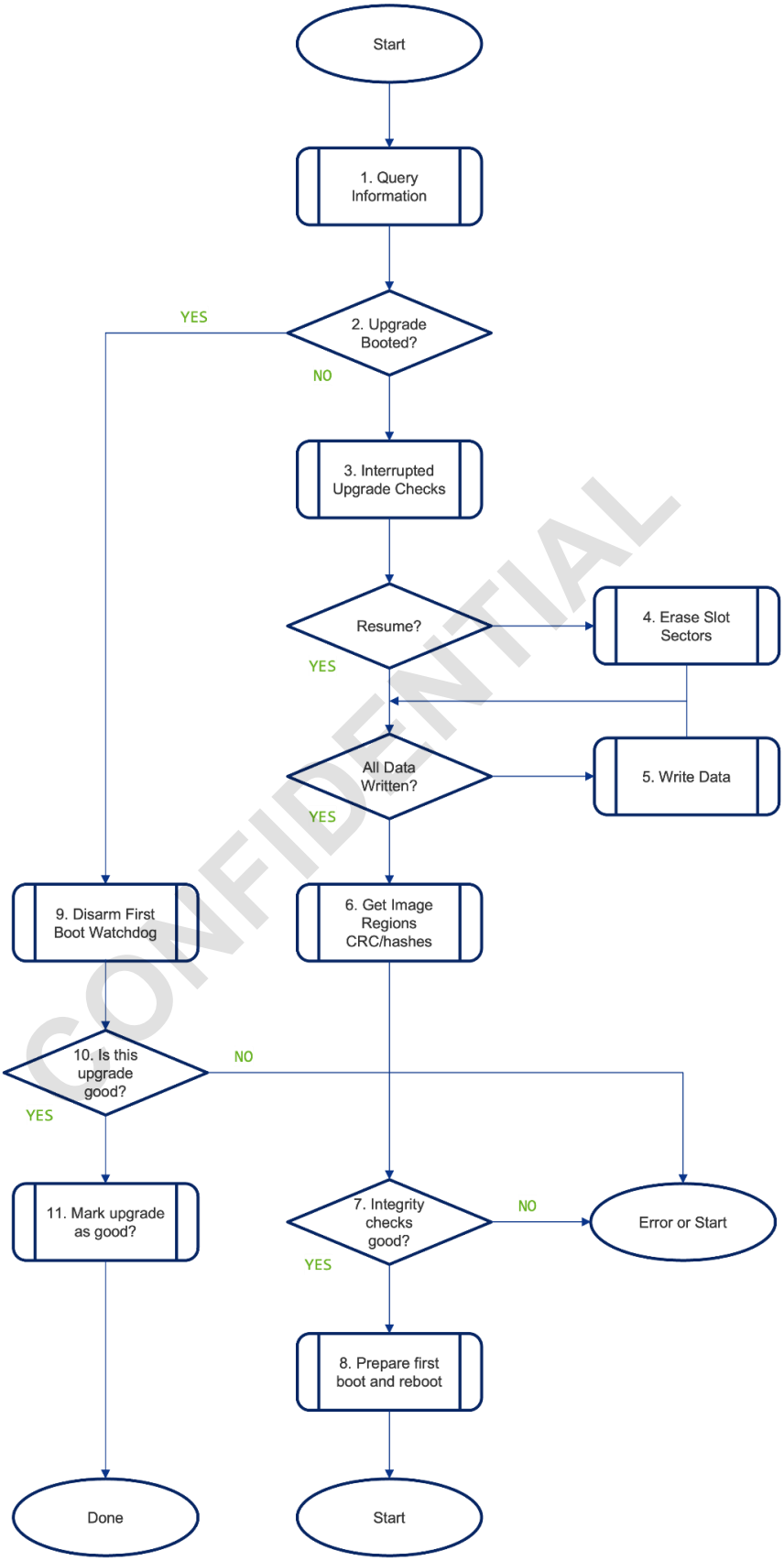
3.1.7 Upgrade State

The server profile does not maintain the state of the upgrade process. Upgrade session state is not maintained between re-connections or power/reset cycles. Any upgrade state can be determined through an inspection of the inactive slot's stored data. The OTA profiles provide methods to aid in assessing the programmed state of the inactive slot.

3.2 Upgrade Process Flow

The high level image upgrade flow is described in the following diagrams. ***Note**: Blocks that are enumerated (1.,2.) have a corresponding detailed description in the Upgrade Flow Step Details section.*

CONFIDENTIAL



CONFIDENTIAL

Figure 3 - Image Upgrade Flow Diagram

3.2.1 Use of a CRC/Hash

The upgrade process makes heavy use of CRC codes to verify the integrity of the data written to RRAM (or external flash). This avoids costly data reads to verify programming. Data in transit however is not CRC/hashed as the transport can provide data integrity checks in transit. Ultimately the determinant of programming success and failure rests on what is actually programmed. The profile provides commands to CRC/hash multiple regions disjointly with one command. The CRC selected is CRC-32 for implementation simplicity, relative speed and error detection.

3.2.2 No Test Boot Option

When MCUBoot is supported on the platform, the No Test Boot option may be indicated in the capabilities field. This option indicates to the host that there is no test boot (first boot) boot flow. In MCUBoot terms this is called the overwrite upgrade strategy. The image stored in the inactive partition is copied (after upgrade image integrity checks) directly to the active partition without a swap. This can improve the upgrade performance as there is no intermediate image swap. The consequence however is that the operation is not reversible and there is no recourse if the image fails to boot.

If the no test boot option is reported in the device capabilities, the first boot results in an overwrite. On the next boot , the active partition will indicate a GOOD status and no first boot stage is entered and no first boot timer (see First Boot Watchdog) will be activated. The application can use other means to determine if the upgrade was successful (version checks in the query information or checking the BLE device information service).

CONFIDENTIAL

3.2.3 Upgrade Flow Steps

The enumerated steps in the previous diagrams are explained below.

1) Client connects to the server profile and queries information regarding the platform and upgrade state of the system.
2) The state of the active slot is checked to determine if an upgrade is in first boot. If the active slot indicates it is a test boot, then the inactive slot contains the recovery image. The upgrade processor will disable all writes to the inactive slot in this case. The client is not allowed to update the contents of the inactive slot during a first boot. The client can only mark the first boot as good.

3) In an interrupted upgrade, the client determines where to resume writing data in the inactive slot. If the client wishes to optimize this process they have the option to employ their own algorithms to determine where the write progress left off. This is done with a combination of CRC commands that walk the inactive partition (slot). The client can get a reasonable assessment of where it left off. Otherwise the client can just start fresh and begin programming the slot.

4) For RRAM storage this is optional. The upgrade processor will report that erase is optional for the underlying memory technology. The OTA protocol supports erase semantics if the client chooses to erase the contents (written to 0xFFs) in the inactive slot. In a hybrid arrangement with external flash, the capabilities will clear the optional erase bit and the query information represents the external flash device managing the inactive partition. External flash must always be erased.

5) Data blocks of the image are written to staging buffers and then buffers are written to RRAM or external flash. If a write operation is interrupted (disconnect or system reset), the client is required to determine where it left off when it reconnects and should make no assumptions about whether the last operation succeeded or not.

6) The client should request CRC/hash values for memory regions that fully validate the upgrade. It is recommended that the regions should cover 4-32K blocks to help debug a cell programming problem.

7) If the integrity check fails the client can retry if required.

8) At this step the client has determined that the slot has been properly programmed. It is now ready to finalize steps to test boot the new upgraded partition. At this step the inactive slot is ready for a test boot and is not considered GOOD until the client has had a chance to reconnect to the image on first boot. The client requests a test boot which will flag, in a non-volatile manner, MCUboot to perform the image swap at the next boot. The system will disconnect and reboot back to MCUBoot.

   ***Note**: The image swap operation can take some time, the client should continue to scan for the device. The amount of time it takes to swap and reboot depends on the size of the upgrade. On a typical image this can range from 30 seconds or more.*

   ***Note on MCU overwrite configurations :** If the no test boot option is reported in the device capabilities, the first boot results in an overwrite. On the next boot the partition will indicate a GOOD status and no first boot stage is entered.*

CONFIDENTIAL

9) A new upgrade has booted through the mechanism in step 8. In the event the client cannot connect back to the OTA server there is an optional first boot watchdog mechanism that will trigger a reset to return back to MCUBoot which will swap the last known good image. The first boot watchdog is disabled after the client can successfully query information from the upgrade processor over Bluetooth LE. Any reboot that occurs while the image is still marked as a test image will result in a reversion back to the last known good image.

10) The client now determines whether the upgraded image is running properly. Note that the upgraded image has been swapped and is now running in the active slot. The upgraded image sees the same NVDS. NVDS is currently not swapped for ATM33 products.

11) This step should ONLY be done when the upgraded image has been swapped to the active slot through a boot back through MCUBoot (step 8). If the active slot meets the client's criteria for a good boot, then the client can now mark the image as GOOD. This is irreversible and unlocks the inactive partition for future upgrades. A command is issued to mark the current active slot (first booted) as GOOD. Any reboots that take place after this point will not revert the image.

3.2.4 First Boot Watchdog

The server profile can be configured with a first boot watchdog. This watchdog can be enabled if the active slot is detected as a first boot. The client has until the watchdog timeout to connect and perform a query information otherwise the watchdog will reboot the platform and MCUBoot will swap back to the last known GOOD image. The watchdog timeout is typically 30 seconds.

Note: In a configuration that does not support test boots, the active partition will not be in a first boot state. Instead the partition is automatically marked as GOOD.

CONFIDENTIAL

3.3 Messages

3.3.1 Types

The upgrade protocol uses the following 4 types of messages:

Command

A control message with variable length parameters. Length of parameters depends on the message. A command always results in a status message with an optional completion message for long running commands.

Status

Immediate status reporting of the last command. The status message may also contain a variable number of parameters which are specific to the command. For example read data or area CRC results.

Completion

Completion messages indicate the completion of long running commands (for example: erase) that report final status separately from a status message. Whereas the command status indicates a command is accepted (valid parameters) with PENDING the completion message reports the final result of the operation.

Write Data

Write messages follow the same format as commands however their delivery and status follow a slightly different flow. Write data messages are fragmented messages to post data to a server-side staging buffer. One or more write data messages will have a single corresponding status and a final completion response. A status is returned on bad parameters, missing fragments (after some timeout) or on a successful transfer right before the programming operation is started. A PENDING status is an indication that the client can now move more data to another staging buffer while programming is in progress. The client can also expect a completion event when programming is complete (either success or failure).

3.3.2 Ports

The following section describes the two port types used for all communications with the server profile. These ports are defined by the Atmosic Data Transfer Service and also used in the Throughput testing profile.

CONFIDENTIAL

Mailbox

The mailbox is reserved for all command, status and completion messages. Transfers to/from the mailbox use the write-with-response and indicate attribute access procedures. This form of transfer, although reliable and avoids potential resource contention issues, is intended for infrequent message exchanges (i.e. control/status). The control protocol is designed such that most messages can fit into a single ATT transfer procedure (single ATT PDU with confirmation down to an MTU size of 23 bytes) however some messages may exceed the indication size limit (ATT_MTU - 3) and require a client initiated read long ATT procedure. Write data transfers to the write buffers can also be performed through the mailbox. This can simplify implementations that do not choose to implement the retransmit logic required over the BULK port.

Bulk

The bulk port provides the fastest and most efficient way to transfer data to waiting write buffers. Data message fragments are sequentially streamed into the write buffers, however, some fragments could be dropped (considered uncommon in most cases). To maximize the performance through the bulk interface, programming and staging buffer transfer should be overlapped when possible.

CONFIDENTIAL

3.3.3 Definitions

Message Format

All fields follow little-endian ordering. For Bluetooth LE, the maximum length of a message is 512 bytes.

Size	Field	Description
1	Msg ID	Message ID is 1 octet (1-255), ID 0 is reserved
n	<parameters>	Variable length parameters (See Message size limits)

Table 2 - Message Format

All messages are encapsulated into the transport's data framing mechanism and thus a length field is not required. The transport layer conveys the length of the full message after decapsulating it and removing any padding. Transport layer is expected to deliver single unfragmented messages.

Name	Value	Description
SUCCESS	0	Success
PENDING	1	Success but the operation is pending and will complete with a completion message
INV_PARAM	2	One or more parameters of the command are invalid
UNSUPPORTED	3	Unsupported command or operation
WRITE_FAILED	4	Failure of a write operation. This status is also used by operations that perform data checks on the integrity of the write (see Copy and Set Upgrade Status).
ERASE_FAILED	5	Erase operation failed.
BUSY	6	Operation could not be completed because the resource is busy.
MISSING_FRAG	7	One or more fragments are missing in the data transfer request
NOT_ERASED	8	The erase operations' post verify operation detected a non-erased byte or a command could not execute because a location was not erased.
INV_PART_STATUS	9	The server encountered an invalid partition status for the requested operation (see Set Upgrade Status)
WRITE_BUF_OVERFLOW	10	The last write data command has overflowed the write buffer
RESP_OVERFLOW	11	The resulting status or completion response overflows the maximum length allowed by the transport. This can happen if the client requests more read data than the transport can deliver in one response. See Message size limits.
CANCELED	12	Operation was canceled
RD_FAILED	13	Read operation failed
INVALID_REQ	14	Invalid Request

Table 3 - Status Codes

CONFIDENTIAL

Command Status

A command status message conveys the success or failure of the last command message. For a command that may require a completion, a PENDING status message indicates the operation has been accepted (parameters validated) and started. Some

command processing ends with a status while others may require both a status followed by a separate completion message (i.e. a long running operation completes).

Size	Field	Description
1	Msg ID	0x01
1	Cmd Msg ID	The message ID of the command this status is for
1	Status	Command processing status code
N	<parameters>	Variable length status parameters (See Message size limits)

Command Completion

A command completion message conveys the success or failure of an operation that requires a delayed indication (long running operation). A long running command returns STATUS_PENDING in the status message and provides a final completion status in a completion message shown below:

Size	Field	Description
1	Msg ID	0x02
1	Cmd Msg ID	The message ID of the command.
1	Status	Final status of the completed long-running operation
N	<parameters>	Variable length status parameters (See Message size limits)

Cancel

This command cancels the current pending operation.

Size	Field	Description
1	Msg ID	0x0F

This message returns a status message with no parameters. The PENDING status will indicate that the cancellation is pending. An operation that was canceled indicates a completion status of CANCELED. If there is no pending operation at the time the

CONFIDENTIAL

command is received the status returns SUCCESS and no command completion will be issued.

Bluetooth LE Considerations

Message size limits

For Bluetooth LE GATT transports, the message size and parameter length limits are listed below:

Message Type	Mail Port (Max. Total Length / Max. Parameters Length)	Bulk Port (Total Length / Parameters Length)
Command	512/511	N/A (not exchanged on this port)
Status	512/509	N/A
Completion	512/509	N/A
Read	See status message, the maximum read length is limited by the maximum parameter length.	N/A
Write Data State	Same as command	ATT_MTU[1] - 3 / ATT_MTU - 4
Write Data Continuation	Same as command	ATT_MTU - 3 / ATT_MTU - 4

CONFIDENTIAL

Pending Message Indication

For Bluetooth LE GATT implementations any message that exceeds the size allowed for an indication (ATT_MTU-3) requires the client to read the message from the port using a read long attribute procedure. To indicate that the client must use this procedure the following message is defined.

Size	Field	Description
1	Msg ID	0xF0
1	length	The length of the pending message that requires a read long procedure.

[1] ATT_MTU - Attribute Maximum Transmission Unit (see Bluetooth Specification)

This is typically used when the command status or completion exceeds the length that is allowed in an attribute indication.

Locking and Unlocking Sessions

Bluetooth LE applications can enter operating states that can interrupt OTA updates. For example, a device may disconnect all links in preparation for low power mode. Devices in low power modes may require external stimulus or user intervention to resume normal operations where connections are permitted. Applications can implement out-of-band techniques (switches, buttons) that place devices in OTA friendly states to ensure a timely update and thus improves the user experience. In the absence of such techniques an over-the-air signaling method can be used to lock a connection that is hosting an OTA session in order to prevent the device from entering states that disrupt the upgrade process. The LOCK/UNLOCK session message signals the client's intent to perform an uninterrupted OTA upgrade. This message is propagated as a software notification to the application. The details of how the application processes the locked/unlocked state is beyond the scope of this document. In general the locked state of the session will prevent the system from entering a state that interrupts the update process. The client can unlock the session and allow the system to return to normal operation upon completing upgrade tasks. Additionally a disconnect as a result of the client or the environment automatically unlocks the session.

Systems that utilize external flash for upgrade storage may require a locked session to ensure that the flash storage device is powered up. Operations can fail if the underlying flash storage device is powered down.

The client can signal locking and unlocking a session using the following messages :

Lock Session

The client can lock the current session to avoid interruptions to the upgrade process. This command returns a status message with no parameters. If the session is already locked the message will fail with INVALID_REQ.

Size	Field	Description
1	Msg ID	0x12

CONFIDENTIAL

Unlock Session

The client can unlock the current session and return the system to normal operation. This command returns a status message with no parameters. If the session is not locked the message will fail with INVALID_REQ. A session is automatically unlocked if the session's connection terminates while it is locked.

Size	Field	Description
1	Msg ID	0x13

Query Information

The client can query the properties of the upgrade server with this command.

Size	Field	Description
1	Msg ID	0x10

This command returns a status message with the following parameters:

STATUS Parameters		
Size	**Field**	**Description**
1	version	Upgrade protocol version information and flags: Bits : 0..3 : Protocol version 0x0 - v1.0 0x1 - v1.1 0x2 - v1.2 0x3 - v1.3 (This version) Bits : 4..7 : RFU
1	capabilities	Capabilities flags Bit : 0 : Stream Mode 1 - Supports write buffer stream protocol[1] 0 - not supported Bit : 1 : Dual write buffer support 1 - Supports dual (ping-pong) buffers[1] 0 - Single buffer Bit : 2 : Resizeable write buffers 1 - write buffers can be resized 0 - write buffers are fixed Bit : 3 : Memory Model 1 - small memory model - this affects the units of measure for the partition size 0 - normal memory model (default) Bit : 4 : Optional Erase 1 - Erase is not required to program 0 - Erase is required to program (default)

CONFIDENTIAL

STATUS Parameters		
Size	**Field**	**Description**
		Bit: 5 : Upgrade Type 1 - MCUBoot (ATM33 and later) 0 - Legacy (ATM2) Bit: 6 : No Test Boot 1 - System does not provide a test boot (no first boot) 0 - System provides a test boot option Bit: 7 : Hybrid Configuration 1 - System is configured with a hybrid partition scheme 0 - System is configured with a normal partition scheme 1. For BLE-GATT Profile : ● If the profile does not support the data stream protocol the client must use the simpler (slower) method over the Mailbox. ● If the profile supports dual buffers, data transfers can overlap with programming otherwise the client must serialize to a single buffer. (**As of this time writing dual buffer support is not supported**)
2	boot_id	Vendor specific board identifier - only understood by the client
1	buffer_size	Write buffer size is 256 byte pages (8 = 4 KB)
1	eraze_size	Erase sector size in 256 byte pages (16 = 4 kB)
1	flash_type	Flash type
2	capacity	Storage capacity in 16 KB units (max 1 GB). The capacity is only for informational purposes
2	part_size	Partition (slot) size in 16 KB units. If the capabilities flag indicates a small memory model this field represents 1 KB units
3	NVDS_data_offset	NVDS copy operations have been deprecated for ATM33 products and this value is set to 0.
1	NVDS_size	NVDS copy operations have been deprecated for ATM33 products and this value is set to 0.
1	active_status	Bits 0..3 Status The following enumerations indicate the partition status 0x0 : reserved 0x1 : The partition is not fully programmed[2] 0x2 : The partition has been programmed[2]

[2] This status is not used when the upgrade type is MCUBoot

CONFIDENTIAL

STATUS Parameters		
Size	**Field**	**Description**
		0x3: The partition is a first boot[3] 0x4 : The partition is good 0xF : The partition has been marked bad Bit 4..7 RFU
1	inactive_status	The upgrade status of the inactive partition or slot (see active_status)
4	Reserved	Reserved
2	mtu	Transport MTU size (Bluetooth LE link MTU size for Bluetooth LE transports)
4	SDK Version	SDK version information in BCD format X.X.X.X
2	ext_caps	Extended capabilities Bits 15..0 RFU

CONFIDENTIAL

Query Version Information

The client can query the properties of the device. If the device does not include the Device Information Service the client can use this method as an alternative.

Size	Field	Description
1	Msg ID	0x11

This command returns a status message with the version information set by the device. The maximum version length is 255 bytes. The actual version information is limited by the transport limits for a status message (See Message size limits).

STATUS Parameters		
Size	**Field**	**Description**
N	Version Information	Version information. A string or data structure defined by the versioning method. If a string is provided, the buffer may or may not contain a NULL terminator. If the device has not set the version information , this field will be empty.

[3] The first boot status supplies only to the active partition.

Write Data Start

The client sends data to the write (staging) buffers as a sequence of messages. The first message is a start message followed by one or more continuation messages. The last fragment in the continuation starts programming. A start message marked as the only fragment also triggers the programming operation. Writes can only be issued for the inactive partition(slot). Write operations cannot be issued when there are outstanding copy or erase operations. Writes assume the sectors have been erased prior.

Size	Field	Description
1	Msg ID	0x20
1	flags	Write flags Bit 0 : Buffer selection (if dual buffer support) 0 - Fill Buffer 0 (default) 1 - Fill Buffer 1 Bit 2..7 : RFU (set to zeros)
4	offset	Starting offset into the inactive partition (slot) where the write buffer will be written to.
2	length	Length of all bytes to be written to the partition
1	count	Count of fragments (1-255) to send (this message is fragment 0). A count value of 0 indicates this buffer contains all the data for the operation and is considered the last fragment.
1	req_seq	Client specific, unique, per-request sequence number (returned in status and completion messages) to track the overall write operation.
<N>	data	Write data up to the remaining length of the first fragment

CONFIDENTIAL

A status message is issued on the reception of all or missing (a timeout) sequences with parameters shown in the following table.

STATUS Parameters		
Size	**Field**	**Description**
1	req_seq	Request sequence copied from the start message
1	seq_good	The highest in-order received sequence number. Can be checked if status is MISSING_FRAG. This is only used if retransmit is supported.

If the status reported is PENDING the parameters/data are accepted and the programming operation has commenced. All other statuses are considered errors and no completion message is expected. The client can wait for a completion message or queue data to the other buffer (if dual buffer mode is supported, **at this time dual buffer mode is not supported)**. Any transfer to a buffer that is being programmed will be rejected with an error.

In stream mode, all reception status indications are deferred. This allows the client to stream the start followed by continuation messages without pausing for status messages. Since it is possible that any message or even all messages including the start message could be lost the status check at the end of the sequence must be checked with a timeout.

The following table shows the parameters for the completion message:

COMPLETION Parameters		
Size	**Field**	**Description**
1	req_seq	Request sequence from the start message

Write Data Continuation

This is a continuation message to deliver a write data fragment. The write buffer offset is required since the start message also delivers data into the buffer. The next continuation message should start where the previous message left off. Also in the event a fragment is re-sent the buffer offset ensures that the data is filled at the correct location in the write buffer. See Message size limits for the upper limit of a fragment.

Size	**Field**	**Description**
1	Msg ID	0x21
1	req_seq	Sequence number for this request. Must match the write start message.
1	seq	Fragment sequence number (1-255). The Write Start message is always sequence 0.
2	buff_offset	Starting offset into the write buffer for the following data
<N>	data	Data up to the remaining length of this fragment

CONFIDENTIAL

This message uses the same status parameters as Write Data Start.

In stream mode, all reception status indications are deferred. This allows the client to stream continuation messages without pausing for status messages. A final check of status with a timeout is required at the end of the sequence.

Last Write Status

This message retrieves the last write status. This status can be used to determine the cause of a status timeout during a stream mode write buffer sequence.

Size	Field	Description
1	Msg ID	0x23

The message returns a status message with the following parameters:

STATUS Parameters		
Size	**Field**	**Description**
1	req_seq	Request sequence tag copied from the start message. In stream mode if the request sequence reported here is different than the expected request sequence then the write start message was likely dropped
1	seq_good	The highest in-order received sequence number from the last req_seq tagged sequence. In stream mode this value can determine if any intermediate fragments (WR CONT messages) were dropped
1	t_count	Fragment count. In stream mode this is only valid if the start message was received.

Read Data

The client can read data in either partition. This command is not optimized to read large amounts of data and intended for small extractions useful in debug or inspection (ex: inspecting NVDS).

Size	Field	Description
1	Msg ID	0x30

CONFIDENTIAL

1	flags	Read flags Bit 0 : Partition selection 0 - Inactive Partition or Slot (default) 1 - Active Partition or Slot Bits 1..7 : RFU (set to zeros)
4	offset	Offset in the partition to read
2	length	Length of data to read (see Message size limits)

In a hybrid or MCUBoot configuration, access to the active partition is disallowed and the operation will return with a status of INVALID_REQ.

The message returns a status message with the following parameters:

STATUS Parameters		
Size	Field	Description
N	Data	Data read from the offset to (offset + length) (see Message size limits)

Get Programmed State

The client can obtain CRC or erase state information across multiple block ranges.

Size	Field	Description
1	Msg ID	0x31
1	flags	CRC/ Erase Verify flags Bit 0 : Partition selection 0 - Inactive Partition (default) 1 - Active Partition Bit 1 : Mode 0 - CRC calculation 1 - Erase state Bits 2..7 : RFU (set to zeros)
N	range_desc	Range descriptors repeated <N> times (see below)

In a hybrid or MCUBoot configuration, access to the active partition is disallowed and the operation will return with a status of INVALID_REQ.

Range Descriptor		
Size	Field	Description

CONFIDENTIAL

4	Offset	Offset in partition
4	Length	Length of data to check (32 KB max for CRC)

For Bluetooth LE the maximum number of range entries is limited by the maximum command parameter size (See Message size limits).

This command returns a status of PENDING if parameters are accepted and the operation has been scheduled. A final command completion message is issued with the following parameters:

STATUS Parameters		
Size	**Field**	**Description**
4*N	CRC or erase state for range repeated N times	If CRC : Bits 0..31 : The 4-byte CRC value If erase state: Bits 0..1 : Erase state : 0 = reserved 1 = erased 2 = not erased 3 = reserved Bits 2..31 : RFU (See Message size limits)

CONFIDENTIAL

Erase Sectors

The client can erase sectors in one or more regions in the inactive partition/slot. The client can optionally request an erase verification step that can report whether any region contains a non-erased byte. This is reflected in the command completion status as NOT_ERASED. Erase operations cannot be issued if there are pending write/erase operations. Erase is not required if erase is optional. For RRAM based systems this command will fill the sector with 0xFFs emulating a flash erase.

Size	Field	Description
1	Msg ID	0x31
1	flags	Erase flags Bit 0 : Erase Verify 0 - Do not perform erase verify (default) 1 - Perform erase verify Bit 1..7 : RFU (set to zeros)

Size	Field	Description
N	range_desc	Sector descriptor repeated <N> times (see below)

This command returns a status message with no parameters. If the status is PENDING, the erase parameters have been accepted and the erase operation is pending. Erase operations cannot be overlapped.

Set Upgrade Status

This command sets the upgrade status for the inactive or active partition/slot. The PROGRAMMED status is applied only to the **inactive** partition in order to allow a first-boot restart where the newly programmed partition is swapped as the **active** partition. The GOOD status only applies to the active partition that is operating in first boot mode. Marking an active partition that is already GOOD returns an error. The client must determine whether the new image operating in first boot mode qualifies as a good image and then and only then set this status. An **inactive** partition CANNOT be marked GOOD. The BAD status can only be applied to an **inactive** partition. In order to set a BAD status the client must reboot the system into the last known GOOD partition and mark the **inactive** partition. This status can be used to quickly mark an upgraded image in the inactive partition that has not properly booted into the first boot stage. PROGRAMMED and BAD statuses only apply to non-MCUBoot upgrade types. Setting these statuses on MCUBoot upgrade types is ignored. It does not affect how MCUboot swaps images.

In a system that reports the No Test Boot option, setting the upgrade status to GOOD is not required and can be skipped. Setting the upgrade status to good while it reports a GOOD status has no effect.

Size	Field	Description
1	Msg ID	0x50
1	status	Partition/Slot status to set 0x0 - Reserved 0x1 - Reserved 0x2 - Set the ***inactive*** partition as PROGRAMMED[4] 0x3 - Set the ***active*** partition as GOOD 0xF - Set the ***inactive*** partition as BAD[2]

[4] No-opts when the upgrade type is MCUBoot.

CONFIDENTIAL

This message returns a status message with no parameters. The status will indicate whether the desired status could be applied to the targeted partition.

Reboot

This command allows the client to remotely reboot the system to revert to the last known GOOD partition or validate the first boot of a new upgrade. When the “No Test Boot” option is set in the device capabilities, the first boot reboot type indicates to the device that the upgrade should be applied, however no first boot state will be entered on the next boot. The upgrade is considered GOOD as soon as it is applied since the system does not provide a test boot option.

Size	Field	Description
1	Msg ID	0x51
1	type	Reboot type 0 - Normal reboot 1 - Perform first boot[5] of the inactive partition/slot

The reboot returns a status with no additional parameters, however the status may not arrive before the system reboots.

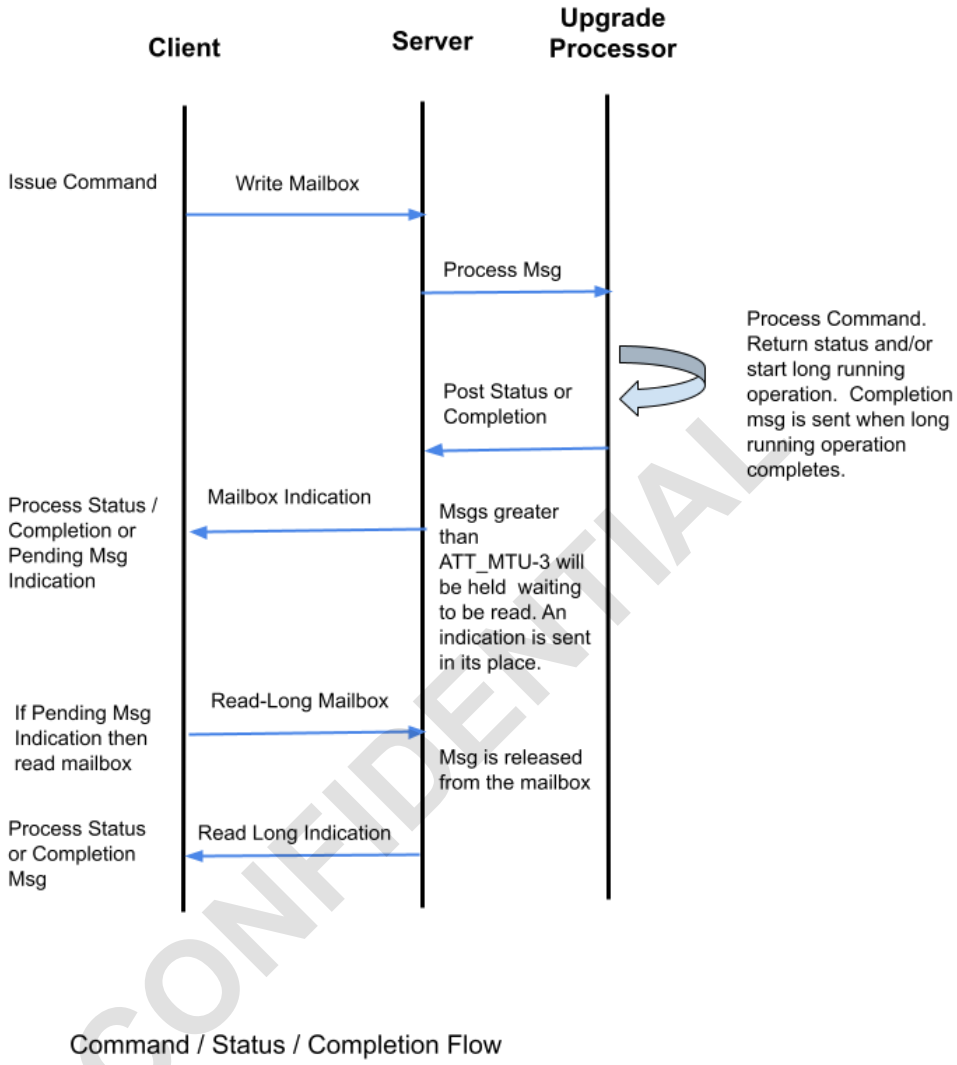
CONFIDENTIAL

3.4 Message Flows

The following section provides flow diagrams for commands, status, completions and data. See Figure 4.

[5] For non-MCUboot types, the first boot reboot type should only be requested if the inactive partition has been marked as PROGRAMMED (see Set Upgrade Status).

3.4.1 Control



CONFIDENTIAL

Figure 4 - Command/Status/Completion Flow Diagram

3.4.2 Write Data

This is the default flow that uses the mailbox port to transfer data to the write buffers. All start and continuation messages report status as mailbox indications. The client must wait for the status before issuing additional messages. This increases overhead but provides reliable and easily debuggable message sequencing. See Figure 5.

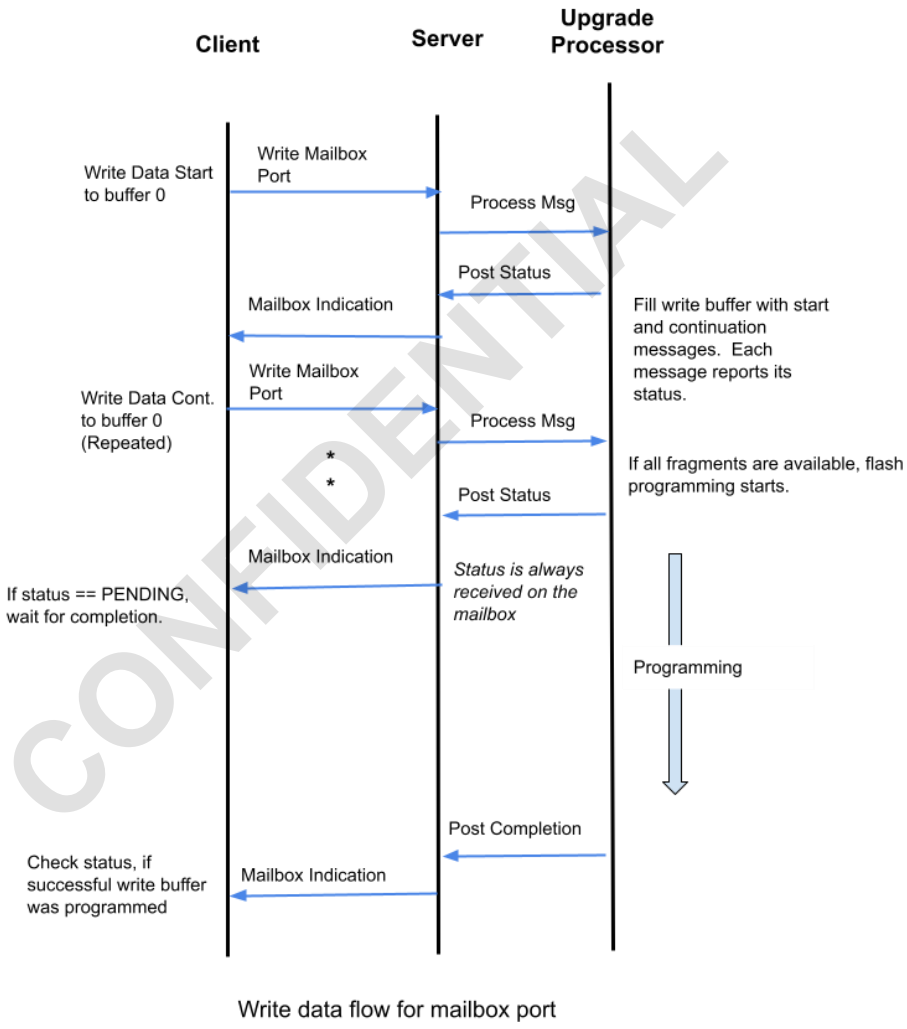


Figure 5 - Write Data Flow for Mailbox Port

3.4.3 Write Data Stream Mode

This flow uses the bulk port to stream write start and write continuation messages to fill the write buffer. This flow uses write-with-no-response GATT procedures and the

CONFIDENTIAL

payload of messages is limited by the negotiated MTU size in addition to the characteristic size . In this flow all intermediate status messages for each start and continuation message regardless of success or failure are omitted with the exception of the final message (start or continuation) that results in programming (i.e. the buffer is filled and ready for programming). In this mode the entire sequence of messages is considered one request and a final status is reported. The status is always reported on the mailbox port. In some client platforms and under certain conditions, the Bluetooth LE stack may optionally drop GATT write-no-response messages that do not require confirmation. This introduces the possibility of one or more (and possibly all) messages in a sequence to be dropped. This requires the client application to employ a timeout when expecting a status at the end of a buffer fill sequence. A client can expect a final status that indicates programming has begun or receive no status at all (timeout) indicating that one or more messages in the sequence was dropped including the possibility of dropping the start message. This is illustrated in Figure 6:

CONFIDENTIAL

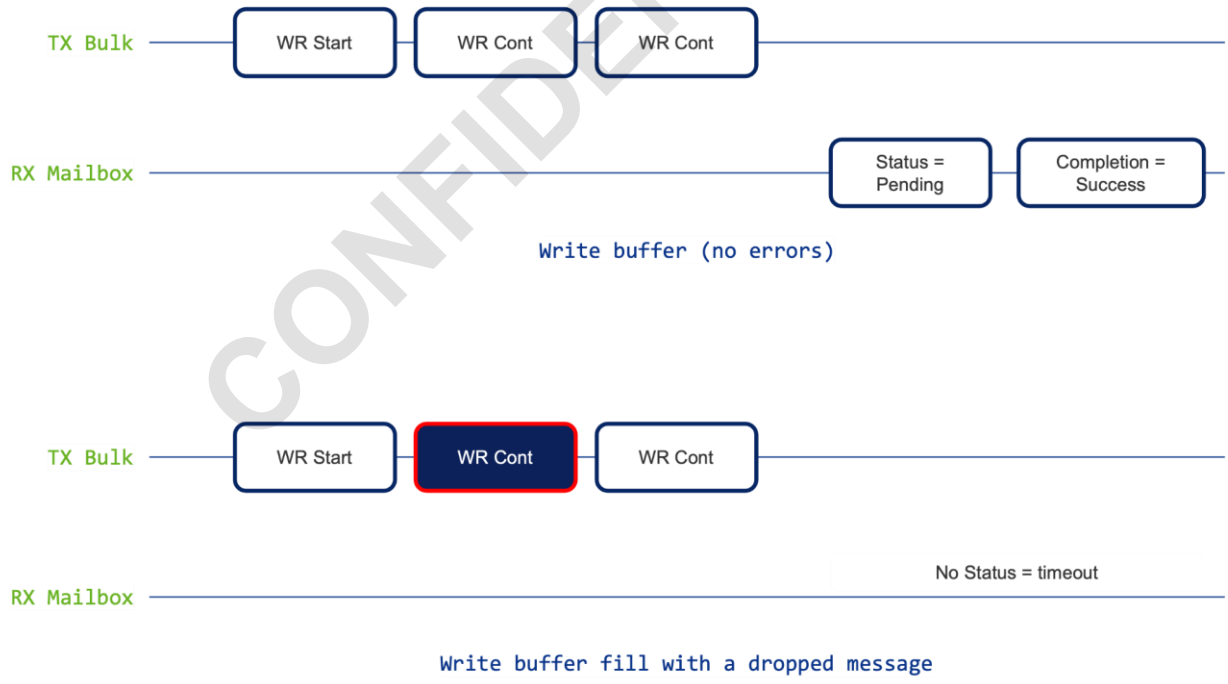


Figure 6 - Write Buffer Fill with a Dropped Message

The use of a timeout is required in the event the entire sequence is dropped. The timeout should be selected based on the number of messages (start + continuations) multiplied by the supervision timeout. For example, given a typical 100ms link

supervision timeout and 5 messages (1 start and 4 fragments), the timeout for the status should be set to at least 500 ms.

In the event of a timeout the application can retry the buffer write sequence or alternatively switch to writing via the mailbox. The Last Write Status command can be issued to get the last write stream status to determine the cause of the timeout.

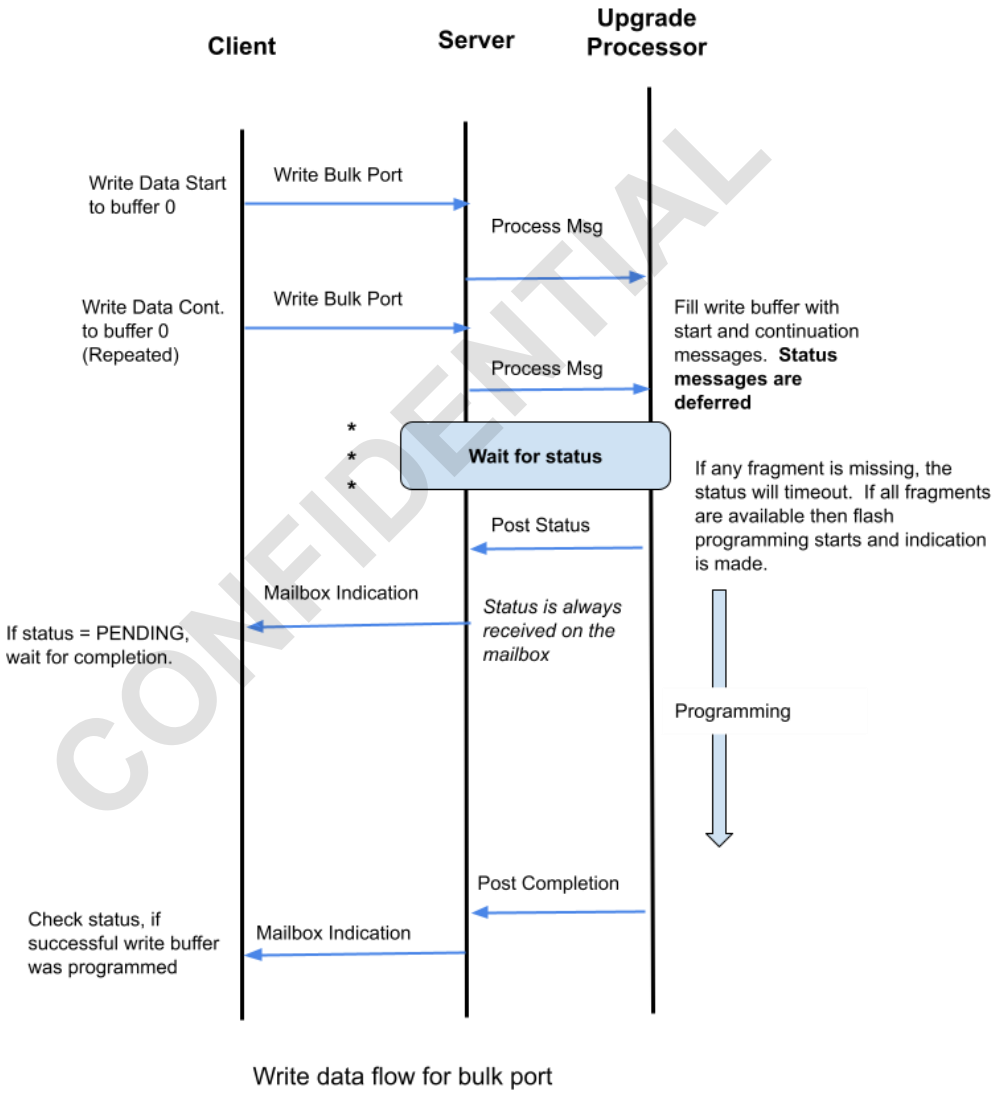


Figure 7 - Write Data Flow for Bulk Port

CONFIDENTIAL

4. Developers Guide

4.1 Installing the profile

The following code snippets can be found in the TPUTP_server example. The following code shows the installation of the profile with default example settings using the ble_otaps library module :

```
// Enable the OTA Service

    static struct otaps_cfg const ota_config = {
    .wr_buffer_size = ATM_TPUTP_WR_BUFFER,
    .board_id = ATM_TPUTP_BOARD_ID,
    .fboot_timeout = ATM_TPUTP_FIRST_BOOT_TIME,
    .ver_info = APP_DIS_FIRM_REV_STR,
    .ver_length = APP_DIS_FIRM_REV_STR_LEN,
    };
    static struct otaps_param const otaps_parm = {
    .sec_lvl = OTA_NO_AUTH,
    .cfg = &ota_config,
    .ota_lock_ind = server_ota_lock_ind,
    };
    atm_gap_prf_reg(BLE_OTAPS_MODULE_NAME, &otaps_parm);
```

The OTA configuration data is defined in the otaps_cfg structure. This structure is passed to the profile via atm_gap_prf_reg().

4.1.1 Board Identifier

The board identifier is specific to the customer's design and only understood by the client profile. This value can be hard coded by the application or read from tags stored in system configuration data.

4.1.2 Write Buffer Size

This value indicates the maximum size of the write buffer to allocate. The recommended value is at least 256 bytes. The application must determine a suitable value given its memory constraints.

CONFIDENTIAL

4.1.3 First boot timeout

If this value is non-zero, the profile will start a first boot watchdog when the active partition's first boot is detected. The time value is defined in scheduling time units (10 ms). The client must connect and send any message (i.e. Query Information) to stop the watchdog. Failure to do so will cause an automatic system reboot and the system will boot from the last known good partition.

4.1.4 Version String

The application can supply optional version information that can be passed to the OTA client. The example shows the version information set to a human readable version string. This option is provided for systems that may not implement the standard Bluetooth DIS.

CONFIDENTIAL

4.2 Changing the Service UUID

The UUID value is constant value that requires a modification to the profile source code. This is not run-time configurable. Locate the profile source module: profiles/otap/otaps/otaps.c.

The constant definition is shown below:

```
static const uint8_t otaps_uuid[ATT_UUID_128_LEN] = OTAP_SERVICE_UUID;
```

Where the array is defined below showing the current default value:

```
#define OTAP_SERVICE_UUID {0x62, 0x90, 0x5c, 0x36, 0xb7, 0xc4, 0xdc, 0x8f,
0x82, 0x41, 0xb5, 0x75, 0x39, 0x8d, 0xe7, 0xa0}
```

4.3 Configuring the Slot Sizes

Please refer to the SDK Users Reference Manual available on the Atmosic Support website on how to size and configure partitions and slots.

4.4 Cleaning Upgrades

The upgrade state is stored in the image trailer. When installing a new image via the debugger this area is cleared to a known state. This will prevent an inadvertent swap due to stale information in the image trailer.

4.5 Testing with a Non-Upgradeable Partition Scheme

During the early stages of development the application may be built with a non-upgradeable scheme to use a full debug version of the Bluetooth LE stack. Although the application will contain a working OTA profile and upgrade processor, this build variant will disable upgrade support. You can still connect to the OTA server profile and query the device. The system will report the following console message in a debug build:

```
WARNING: MCUBOOT not configured, upgrades disabled
```

If an information query is performed the debug field in the query information parameters will contain the following token : 0xDEADBEEF

In this mode, all operations that can mutate RRAM are disabled and will return an error This includes erase, write, copy and setting the upgrade state.

CONFIDENTIAL

Reference Documents

Title	Document Number
Over The Air Update Service User Guide for ATM2/ATM3	ATMx221-UGOTA-0054
ATM33_e Series MCUboot Application Note	ATM33_e-ANMB
SDK User Reference Manual for ATMx2xx-x1x	ATMx2xx-x1x-SWUR
ATM33/e Wireless SoC Series SDK User Reference Manual	ATM33_e-URMSDK

CONFIDENTIAL

Revision History

Date	Version	Description
March 17, 2023	0.52	Update for SDK 5.4.0.1 - Overview, Features, New to Version 1.4, Figure 1 - OTA System Block Diagram, Upgrade Processor, Storage Subsystem, RRAM / External Flash, Upgrade Flow Steps, First Boot Watchdog, Locking and Unlocking Sessions, Unlock Session, Read Data, Get Programmed State, Set Upgrade Status, Reboot. Added No Test Boot Option. Renamed to OTA Update Server User Guide for support of ATM2/ATM3/ATM33.
February 3, 2023	0.51	Updated New to Version 1.3, removed Copy Data section, corrected typos.
May 9, 2022	0.50	Initial version created

CONFIDENTIAL

ATMOSIC TECHNOLOGIES – DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2022-2023 by Atmosic Technologies. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders.

CONFIDENTIAL

Atmosic Technologies | 2105 S. Bascom Ave. | Campbell CA, 95008
www.atmosic.com